

0.59.3. La fonction `aller_retour()` — aller et revenir

`#include <aller_retour>` : `aller_retour(x, y, tuile, pasX, pasY, vitesse, fois)` va et revient, « `fois` » fois.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `aller_retour(...)` mène la tuile jusqu'en $(x + \text{pasX}, y + \text{pasY})$, puis la **ramène**, autant de fois que `fois`. Elle finit à sa place de départ.

La ligne à écrire : `#include <aller_retour>`.

Ses arguments : la place de départ, la tuile, le trajet en x et en y, la vitesse, le nombre d'allers-retours.

Ce qu'elle coûte : environ **460 octets**, plus la police entière.

Essaie : `4, 4` : en diagonale.

On retrouve `aller_retour` dans 9 leçons

[0.60](#), [0.60.1](#), [0.60.2](#), [0.61](#), [0.61.1](#), [0.61.2](#), [0.64](#), [0.64.1](#), [0.64.2](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1308 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage – la mise en route de la console
<code>aller_retour()</code>	400 o	← <code>main()</code> – une fonction fournie par le langage
<code>main()</code>	37 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← <code>image()</code> – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← <code>EffacerCases</code> – attend de pouvoir écrire une case
AdresseCase	10 o	← <code>poser()</code> , <code>effacer()</code> – trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← <code>effacer()</code> – efface des cases
CopierTuiles	18 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
Tuiles	704 o	← <code>CopierTuiles</code> – la police entière

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

<code>poser()</code>	× 3	39 o sur place, dans <code>aller_retour()</code> + <code>AdresseCase</code> (10 o)
<code>effacer()</code>	× 2	30 o sur place, dans <code>aller_retour()</code> + <code>AdresseCase</code> (10 o), <code>EffacerCases</code> (12 o)
<code>image()</code>	× 3	9 o sur place, dans <code>aller_retour()</code> , <code>main()</code> + <code>AttendreImage</code> (28 o)

```
Tes fonctions appelées :  
    aller_retour() × 1    30 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

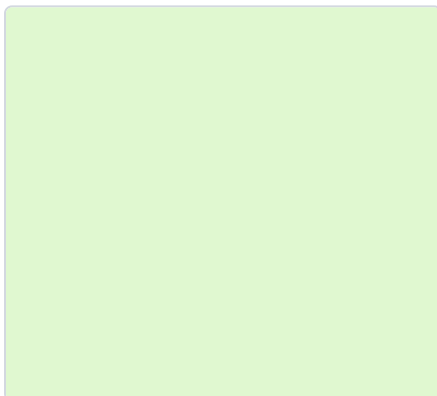
```
// ---- #include <aller_retour> : aller et revenir ----  
  
#include <aller_retour>    // aller_retour() : un trajet, et retour  
#include <ALPHABET>
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

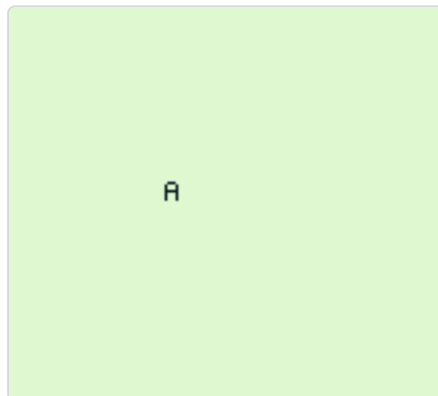
```
int main() {  
    aller_retour(5, 8, ALPHABET[0], 6, 0, 100, 2);    // de (5, 8) à (11, 8), deux fois  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



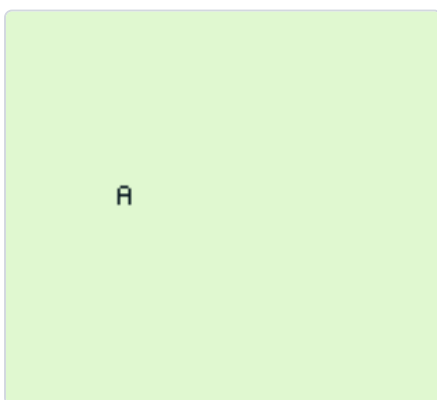
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un A qui va de gauche à droite et revient, deux fois.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A va jusqu'en (11, 8), deux fois (2 fois)

✓ et finit à sa place de départ

Ensuite : **0.60. Aller et revenir : `aller_retour`** — chapitre 0 : Avant tout. · `gameboy3` — écrire une cartouche Game Boy en C++